

Week 2: Statement, Data Types and Variables

1. Statements

A **statement** is a command given to the computer that instructs the computer to take a specific action, such as display to the screen, or collect input. A computer program is made up of a series of statements.

A statement can be a **simple statement** that does not contain other statement or a **compound statement** that contains other statement. In this tutorial we will focus on simple statements, and other types of statement will be covered in subsequent tutorials.

The simplest kind of statement in C is an expression (followed by a semicolon, the terminator for all simple statements). Its value is computed and discarded.

Examples:

```
1. x = 2;          /* an assignment statement */
2. x = 2-3;        /* another assignment statement */
3. 2+3;           /* has no effect---will be discarded by smart compilers */
4. printf("hi");   /* a statement containing a function call */
5. root2 = sqrt(2); /* an assignment statement with a function call */
```

Simple statements are also known as **expression statements**. The above examples are simple statements which perform a well-defined unit operation such as addition, subtraction, square root, printing a message etc. Every statement is ended with a semicolon (;).

Some of the operations that can be performed in a simple statement are given below. In this tutorial, we will focus more on arithmetic operators and assignment operator. As we go along, other forms of operators will also be introduced.

Operators in C

	Operators	Type
Unary Operator →	++, --	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Rational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator

Comment Statement:

Tagging your program with useful message is a good practice, for better understanding and future reference. When the program become bigger, such messages become handy. There are two ways by which you can include comments in your program.

- a) `/* ... */`: Type your message with the `“/*”` and `“*/”`. You can include comments of any length, any number of lines.
- b) `//`: write your comment after `“//”`. You can make one line comment.

2. Variables

In the above example, we have used a symbol `“x”`. This is known as a variable. A **variable** is basically **the name of a memory location/container that we use for storing data values**. For example, if we want to perform addition of two integer numbers and store the sum, we basically need three variables that can hold integer values.

Let us say, the numbers to be added are stored in two variables names **a** and **b**, and the sum of the values stored at **a** and **b** is stored in another variable called **c**. This can be realized by the following statement.

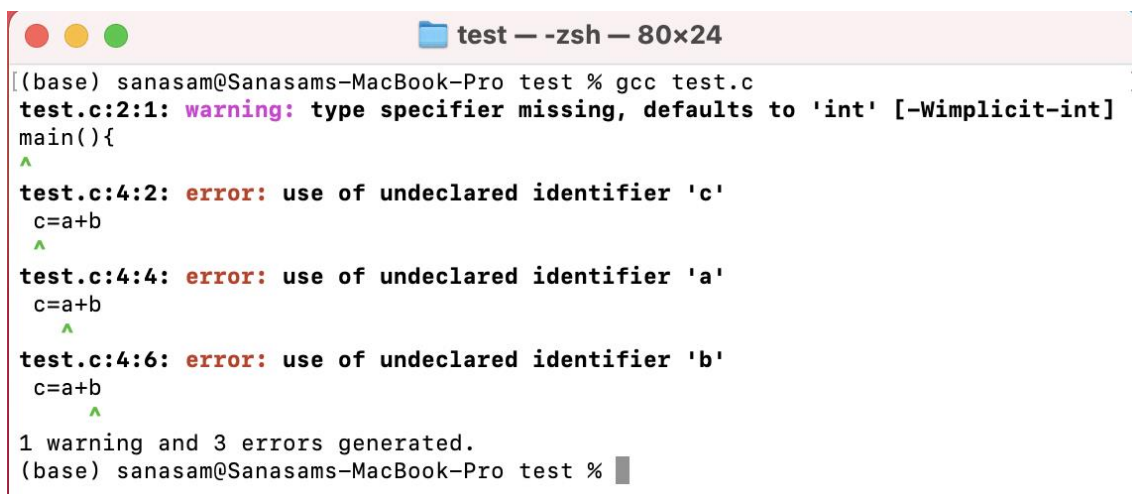
```
c = a + b;
```

Let us write an example program to perform the above operation.

```
#include <stdio.h>

main(){
    c = a + b;
}
```

If you compile the above program, you will see compilation errors like below. Understanding different type of errors, and resolving them is the key in learning programming language. We encourage you to intentionally introduce different nature of errors and see their response.



```
test — -zsh — 80x24
(base) sanasam@Sanasams-MacBook-Pro test % gcc test.c
test.c:2:1: warning: type specifier missing, defaults to 'int' [-Wimplicit-int]
main(){
^
test.c:4:2: error: use of undeclared identifier 'c'
  c=a+b
  ^
test.c:4:4: error: use of undeclared identifier 'a'
  c=a+b
  ^
test.c:4:6: error: use of undeclared identifier 'b'
  c=a+b
  ^
1 warning and 3 errors generated.
(base) sanasam@Sanasams-MacBook-Pro test %
```

Note that compiler has found three errors at line number 4. In the above screen, it shows that inside the function named **main()**, at the line number 4, and token number 2, 4, and 6, there are errors. The nature of the error is “**undeclared identifier**”. It means the variables a, b, c are not declared. The type of message shown by the compiler may be different for different version/type of C compiler used – Turbo C, Borland C, cc, gcc etc. You should check, which compiler you are using.

It basically means that all the variables should be first declared before using them.

Naming a Variable:

- A variable name can start with only alphabet, and underscore.
- A variable should contain only numbers, alphabets, or underscore.
- No **keywords** will be considered as a variable. C program reserves few sets of names such as main, int, char, float etc. Such reserved names can not be used as variable name.
- The variable length is limited to 31 characters.

3. Data Types

When a variable is declared, it should be associated with a **datatype**. The datatype in C defines the amount of storage allocated to variables, the values that they can accept, and the operation that can be performed on those variables.

For example, if you want to define a variable named **x** which can hold an **integer value**, we can declare it as follows.

```
int x;
```

In the above example, the **int** indicates the datatype associated with the variable x. The general syntax of declaring a variable is “**<datatype> <variable name>;**”. You can also declare more than one variable in a single declaration. In that case, the variable names are separated by a comma (.). For example, “**<datatype> <variable name 1>, <variable name 2>, <variable name 2>,...**”.

Now, the above example can be corrected as follow.

```
#include <stdio.h>

main(){
    int a, b, c;
    c = a + b;
}
```

When you compile the above program, there is no compilation error. You may see some warning messages. For now, you can ignore the warning messages.

But, what the values of numbers, that is, the values in the variables a and b. We can assign the values of these variables in the program itself using assignment operator (=).

```
#include <stdio.h>

main(){
    int a = 5, b = 7, c;
    c = a + b;

    printf("%d", c);
}
```

In the above example, you initialize a variable in the declaration statement. When you compile the above program and execute, you will see that the program displays 12 on the terminal.

Instead of initializing a variable in its declaration, you can also assign the values in separate statements like below.

```
#include <stdio.h>

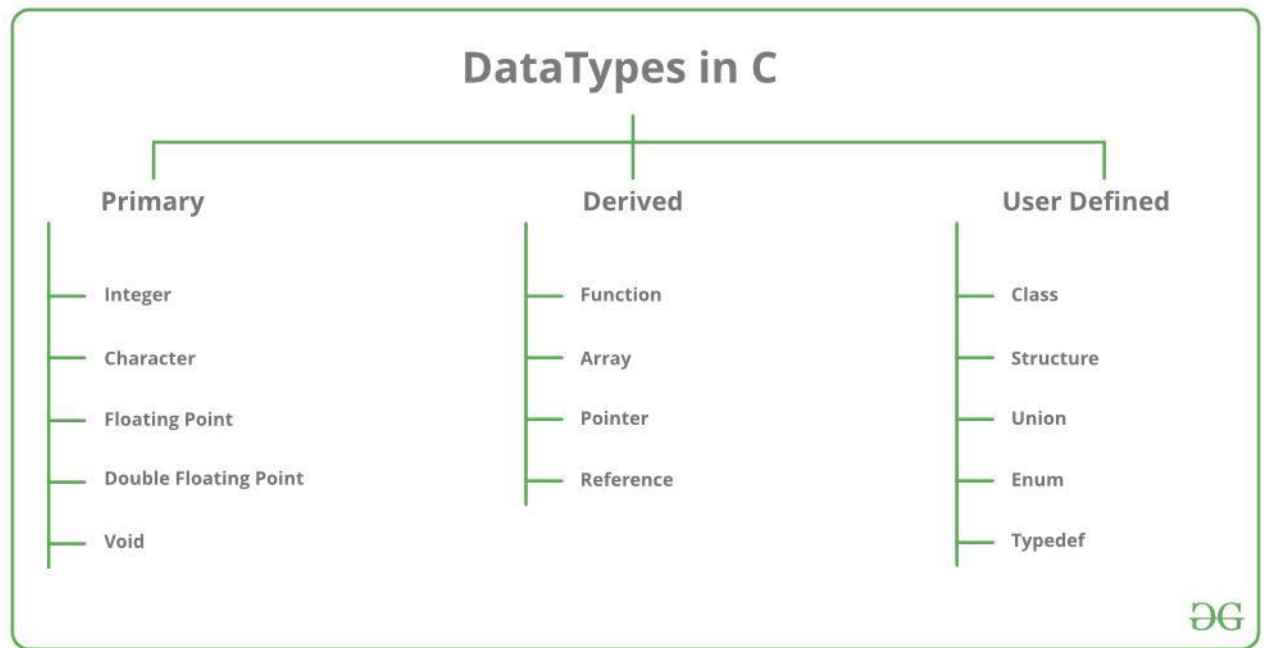
main(){
    int a, b, c;

    a = 5;
    b = 7;
    c = a + b;

    printf("%d", c);
}
```

In the above program, we have introduced a function name “**printf**”. This is a function provided by C compiler to display a message on an output device (Computer Monitor, in our case). We will see more one detailed in the follow up section.

Various kinds of Data Types are supported in C. Some of which are listed below. A datatype can be primary(primitive), derived or user defined. We will focus on primary data types, and other data types will be introduced in subsequent tutorials.



Ref Image: <https://www.geeksforgeeks.org/data-types-in-c/>

The keywords to define the different datatypes, their permissible values, size of memory in byte, and format are shown below.

Data Type	Size (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
int	4	-2,147,483,648 to 2,147,483,647	%d
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	-(2 ⁶³) to (2 ⁶³)-1	%lld
unsigned long	8	0 to 18,446,744,073,709,551,615	%llu

Data Type	Size (bytes)	Range	Format Specifier
long int			
signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
float	4	1.2E-38 to 3.4E+38	%f
double	8	1.7E-308 to 1.7E+308	%lf
long double	16	3.4E-4932 to 1.1E+4932	%Lf
char	1	0 to 255	%c

Ref Table: <https://www.geeksforgeeks.org/data-types-in-c/>

In the following example, the program displays the amount of memory allocated to each datatype. The **sizeof()** is a function which returns the number of bytes allocated to a data type or a variable. The **sizeof** function is defined under **stdio.h** header file. You are encouraged to see for other data types those are not included in the example.

```
#include<stdio.h>

main(){
    printf("%d \n", sizeof(int));
    printf("%d \n", sizeof(float));
    printf("%d \n", sizeof(char));
    printf("%d \n", sizeof(short int));
}
```

You can also see similar results by using variable names. The program can be changed as below.

```
#include<stdio.h>

main(){
    int a;
    float b;
    char c;
    short int d;

    printf("%d \n", sizeof(a));
```

```

        printf("%d \n", sizeof(b));
        printf("%d \n", sizeof(c));
        printf("%d \n", sizeof(d));
    }

```

There is another special datatype called **bool** (for boolean). Boolean datatype is not defined in the standard C compiler. It has been included as extended module under **stdbool.h** header file. A boolean variable can hold a bit of information which is either 0 or 1, OR true or false. An example program is shown below.

```

#include <stdbool.h>

main()
{
    bool a = true;
    bool b = false;

    printf("True : %d\n", a);
    printf("False : %d\n ", b);

    printf("%d", sizeof(a));
}

```

Though a boolean variable conceptually holds 1 bit of information, 1 byte will be allotted to a variable of boolean type (can be seen from the above program), a 1 byte is the smallest unit that CPU cannot address anything smaller than 1 byte.

4. Header

In all the above examples, you might have observed a line “**#include<stdio.h>**”. All the functions, C defined keywords, datatypes those come with C compiler are defined and placed in different modules of C library. The “**stdio.h**” means header for **standard input output**. All the standard input output related modules included in C library are defined in a header file name **stdio.h**.

Let us see the response of compiler without the header file.

```

main(){
    printf("Hello World ");
}

```

While compiling the above program, you will see error message like below. It shows that compile does not recognise the function **printf()**. Likewise you can also test without **main()** function. C is procedural language where execution of the program starts from main function. Every C program should have **main()** function.

```
user@user: ~/cs110/2024/prog
(base) user@user:~/cs110/2024/prog$ gcc test.c
test.c:2:1: warning: return type defaults to 'int' [-Wimplicit-int]
  2 | main(){
    | ^~~~~
test.c: In function 'main':
test.c:3:9: warning: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
  3 |     printf("Hello World");
    |     ^~~~~
test.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'
+++ |+#include <stdio.h>
  1 |
test.c:3:9: warning: incompatible implicit declaration of built-in function 'printf' [-Wbuiltin-declaration-mismatch]
  3 |     printf("Hello World");
    |     ^~~~~
test.c:3:9: note: include '<stdio.h>' or provide a declaration of 'printf'
test.c:3:16: error: stray '\342' in program
  3 |     printf("Hello World");
    |                ^
test.c:3:17: error: 'Hello' undeclared (first use in this function)
  3 |     printf("Hello World");
    |            ^~~~~
test.c:3:17: note: each undeclared identifier is reported only once for each function it appears in
test.c:3:22: error: expected ')' before 'World'
  3 |     printf("Hello World");
    |                ^~~~~~
test.c:3:28: error: stray '\342' in program
  3 |     printf("Hello World");
    |                ^
(base) user@user:~/cs110/2024/prog$
```

The function **printf()** is defined under **stdio.h**. Therefore, if you are using a function, you should first include the corresponding header file. More header file, we will be introduced in the subsequent tutorials as we use more functions of different nature.

5. The **printf** and **scanf** functions

Two important functions in any programming language are the function to send a message to an output device (say, displaying message on the terminal), and read input from the user (or from input device such as keyboard).

printf(): The printf function is used to display messages on terminal. Few examples of using printf() function have been illustrated above. Let us explore few more details.

syntax: int printf(const char *format, ...)

It takes the **format** (it is a string in a defined format) as input, and return in integer as output (the number of character displayed).

The format has the following structure.

This is the string that contains the text to be written to stdout. It can optionally contain embedded format tags that are replaced by the values specified in subsequent additional

arguments and formatted as requested. Format tags prototype is **%[flags][width][.precision][length]specifier**, which is explained below –

Specifier & Output	
1	c Character
2	d or i Signed decimal integer
3	e Scientific notation (mantissa/exponent) using e character
4	E Scientific notation (mantissa/exponent) using E character
5	f Decimal floating point
6	g Uses the shorter of %e or %f
7	G Uses the shorter of %E or %f
8	o Signed octal
9	s String of characters
10	u Unsigned decimal integer

11	x Unsigned hexadecimal integer
12	X Unsigned hexadecimal integer (capital letters)
13	p Pointer address
14	n Nothing printed
15	% Character

Sr.No.	Flags & Description
1	- Left-justify within the given field width; Right justification is the default (see width sub-specifier).
2	+ Forces to precede the result with a plus or minus sign (+ or -) even for positive numbers. By default, only negative numbers are preceded with a -ve sign.
3	(space) If no sign is going to be written, a blank space is inserted before the value.
4	# Used with o, x or X specifiers the value is preceded with 0, 0x or 0X respectively for values different than zero. Used with e, E and f, it forces the written output to contain a decimal point even if no digits would follow. By default, if no digits follow, no decimal point is written. Used with g or G the result is the same as with e or E but trailing zeros are not removed.
5	0

	Left-pads the number with zeroes (0) instead of spaces, where padding is specified (see width sub-specifier).
Sr.No.	Width & Description
1	(number) Minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger.
2	* The width is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.
Sr.No.	.precision & Description
1	.number For integer specifiers (d, i, o, u, x, X) – precision specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. The value is not truncated even if the result is longer. A precision of 0 means that no character is written for the value 0. For e, E and f specifiers – this is the number of digits to be printed after the decimal point. For g and G specifiers – This is the maximum number of significant digits to be printed. For s – this is the maximum number of characters to be printed. By default all characters are printed until the ending null character is encountered. For c type – it has no effect. When no precision is specified, the default is 1. If the period is specified without an explicit value for precision, 0 is assumed.
2	.* The precision is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.
Sr.No.	Length & Description
1	h The argument is interpreted as a short int or unsigned short int (only applies to integer specifiers: i, d, o, u, x and X).
2	l The argument is interpreted as a long int or unsigned long int for integer specifiers (i, d, o, u, x and X), and as a wide character or wide character string for specifiers c and s.
3	L The argument is interpreted as a long double (only applies to floating point specifiers: e, E,

f, g and G).

- **additional arguments** – Depending on the format string, the function may expect a sequence of additional arguments, each containing one value to be inserted instead of each %-tag specified in the format parameter (if any). There should be the same number of these arguments as the number of %-tags that expect a value.

Example:

```
#include <stdbool.h>

main(){
    int a=6, b=5;

    printf("The value of a is %d and the value of b is %d \n", a,b);
}
```

In the above printf statement, the first %d correspond to the variable a, and the second %d corresponds to the variable b. Within the "...", you can type the message you wish to display. The value of the first variable will be embedded at the place defined by first %d, and the value of the the second variable is embedded at the place defined by the second %d. The %d is used because the target variable is an integer. You can use %f for float or %c for character variable. Please refer to the above table for **specifiers of different datatypes in the table above**.

Example:

```
#include <stdbool.h>
#include<stdio.h>
main(){
    int a=6;
    float b=5.5;
    char c = 'x';    // character x is stored in the variable c

    printf("The value of a is %d, the value of b is %f, the value of c is %c \n", a,b);
}
```

The “\n” represent new line. You can check the difference by removing “\n”.

While displaying the values of an integer variable or a float variable, we can also define the number of digits, using the following formats

specifier	display
%d	print as decimal integer
%kd	print as decimal integer, at least k characters wide
%f	print as floating point

%kf	print as floating point, at least k characters wide
%.kf	print as floating point, k characters after decimal point

Example:

```
#include<stdio.h>

main(){
    int a= 5;
    int b= 1818;
    float c = 5.48958123;
    char d = 'x';

    printf("%d \n", a);
    printf("%2d \n", a); // see the space before the digit in the output.
    printf("%d \n", b);
    printf("%2d \n", b);
    printf("%f \n", c);
    printf("%0.4f \n", c);
    printf("%c \n", d);
    printf("%3c \n", d); // see the space before the character in the output.
}
```

scanf(): The scanf function is used read input from input devices (say, keyboard).

Syntax: **int scanf(const char *characters_set)**

Example

```
#include<stdio.h>

main(){
    int a;

    scanf("%d", &a);

    printf("%d", a);

}
```

In the above example, the “%d” specifier is used because the program is going to take an integer value from the keyboard. Also note that an **&** symbol is also placed before the variable. & indicates that the address of the variable a will be passed as the parameter to the function that the variable. **It is a common mistake that beginner often misses to include &. Please give special attention to & while using scanf.**

Please note that when the program executes the scanf statement, execution will stop at this point and wait for entering a value. After entering the value, press Enter key.

You can also take values of multiple variables using a single scanf. An example is given below. The first specifier corresponds to the first variable, second specifier corresponds to the second variable and so on.

Example:

```
#include<stdio.h>

main(){
    int a;
    float b;
    char c;

    scanf("%d %f %c", &a, &b, &c);

    printf("%d", a);
    printf("%f", b);
    printf("%dc", c);

}
```

6. Scope of a variable

Every variable defined in your program has its scope. There are basically two types of variables – global, and local. All the example used in the above examples are local variable. We will explore global variables in the subsequent tutorial.

All the variables defined inside the main() function are **valid only within the main function**. You cannot use the variable outside the main function. As we go along, we will get to know more about the scope of variable. In this tutorial, we will focus scope of variables defined inside the main function.

Consider the following example. Two variables a and b are defined inside the main function. However, the variable b is defined inside a {...} a block. A logical block can be defined using {...}. any variable defined within a block is valid only within that block.

Therefore, last usage of b is an error. See for yourself.

Example.

```
#include<stdio.h>

main(){
    int a=5;

    printf("%d",a);

    {
        int b=10;
        printf("%d", a);
        printf("%d", b);
    }
```

```
printf("%d", a);  
printf("%d", b); // error  
  
}
```

Happy Learning!